

jiRAG: Grounded, Cited and Abstaining Question Answering over a Jira Ticket Corpus with Multilingual Retrieval, QLoRA Adaptation and a Controlled 2×2 Ablation

Ilia Abramovich • Sagi Bondarenko • Ouriel Ohayon

Google and Reichman AI Tech School — Machine Learning and Deep Learning Course
Applied Language Models — Group Project

https://github.com/Sagib/rag_project_google_reichman

Abstract—A manager asking whether an incident ever occurred, and how it was resolved, is asking a question whose answer already exists in Jira — buried in prose across a year of tickets. A language model that answers such a question fluently but without proof is worse than no model at all. We present *jiRAG*, an end-to-end retrieval-augmented generation system over a contract-validated corpus of 1,000 anonymized Jira-style tickets, which answers only from retrieved tickets, cites each material claim as `tckt-XXXX`, and refuses when the evidence does not cover the question. The pipeline covers corpus validation, a leakage-safe stratified 750/150/100 split, a controlled comparison of three sentence encoders, a persistent FAISS vector store, cross-encoder reranking, versioned grounding prompts, and two controlled QLoRA experiments on Gemma 4. Selecting `multilingual-e5-base` raised overall retrieval Hit@1 from 0.682 to 0.909 and Hebrew Hit@1 from 0.000 to 0.833 at no cost in English quality; reranking then lifted validation Hit@1 from 0.818 to 0.909 and frozen-test MRR@5 from 0.972 to 1.000. A 2×2 ablation on the frozen test benchmark separates the two contributions cleanly: retrieval is the grounding mechanism, moving Gold citation recall from 0.000 to 0.972 and eliminating invented citations (0.25 $\rightarrow$ 0.00), while QLoRA improves how retrieved evidence is used — expected-point coverage 0.986 $\rightarrow$ 1.000, citation precision 0.972 $\rightarrow$ 1.000, ROUGE-L 0.262 $\rightarrow$ 0.321 — at the cost of collapsing no-answer abstention from 1.00 to 0.00, which we restore with a deterministic agent evidence gate rather than by re-tuning after seeing test results. The system is delivered to non-technical users through a role-aware web portal, `cladding-labs-os.com`, from which a manager can query the assistant and open the underlying Jira board.

I. INTRODUCTION AND MOTIVATION

Operational teams accumulate Jira tickets faster than anyone reads them. The information a manager needs — did we ever expose supplier pricing, what caused it, was it actually fixed — is present, in free-text prose, distributed over hundreds of records with inconsistent wording. Keyword search fails because symptoms are described differently each time; reading the backlog does not scale; and a general-purpose language model asked the same question will answer fluently from parametric knowledge it does not have, which in an operational setting is the worst of the three options.

Retrieval-augmented generation [1] is the standard response: retrieve the relevant records, then generate only from them. But retrieval alone does not make a system trustworthy. The generator must cite what it used, must distinguish a verified fix from a workaround rather than treating a *Done* status as proof, must answer a Hebrew question over an English corpus, and must refuse when the retrieved evidence does not cover the question. Those four behaviours, not fluency, are what an inspector of the system would check.

Our objective is therefore twofold. First, build a complete, reproducible pipeline from a validated ticket corpus to a deployed manager-facing assistant that is grounded, cited, read-only and multilingual. Second, treat the two adaptation mechanisms avail-

able to us — retrieval and parameter-efficient fine-tuning — as an experiment rather than a stack: a controlled 2×2 ablation varying only the presence of retrieved context and the presence of a QLoRA adapter, on a benchmark frozen before it was opened. The result is a decision made from evidence, including evidence that argues against the system we selected.

II. CORPUS CONSTRUCTION AND VALIDATION

A. From five seed tickets to a balanced corpus

Real ticket text cannot be published. Five anonymized production tickets were therefore used as seeds, and LLM data augmentation expanded them into a balanced corpus of **1,000 tickets** — ten technical families of one hundred tickets each, every ticket written as seven fixed sections (CONTEXT, ISSUE OR REQUEST, EXPECTED, ACTUAL, INVESTIGATION, RESOLUTION, VALIDATION) of 90–220 words. Augmentation was prompted per family and per solution state so that balance was designed rather than discovered.

Every generated ticket then passed **two independent reviews**: a manual plausibility read that removed implausible records, and a second model acting as judge over the full list, scoring duplication, section completeness, and contradictions between workflow status and claimed resolution. The corpus is synthetic by construction, and we state that as a limitation rather than a footnote (Section X); it was the only way to ship a public repository with realistic ticket depth.

B. The dataset contract

Nothing downstream runs until Section 1 of the notebook validates the corpus against an explicit contract, `dataset_contract.json v1.2`: expected row count, expected raw header, per-solution allowed statuses, a 90–220 word description bound, and a canonical SHA-256 of the approved file. A separate scan flags e-mail addresses and credential assignments as errors and IPv4 literals as warnings. The raw file is re-hashed at the end of the section to prove it was not modified in between.

One contract rule does real work later. Status is *constrained by* solution state: a *verified* ticket must be *Done*, and an *unresolved* ticket can never be *Done*. That constraint is what allows a later guardrail to detect an answer claiming a permanent fix about a ticket whose evidence supports only a workaround.

C. Operational profile

Figure 1 summarises the corpus. It contains 998 unique components over 1,000 tickets, 641 *Done* against 359 open, and is deliberately skewed toward defects (746 *Bug*, 183 *Task*, 71 *Story*) and mid-band urgency (491 *Medium*, 331 *High*, 104 *Low*, 74 *Highest*). Solution state is fixed by design at 550 *verified*, 150 *partial*, 150 *workaround* and 150 *unresolved*, and is spread evenly across the ten families, so no family is silently easier than another. Figure 1c shows the contract rule in effect: the *verified* row is

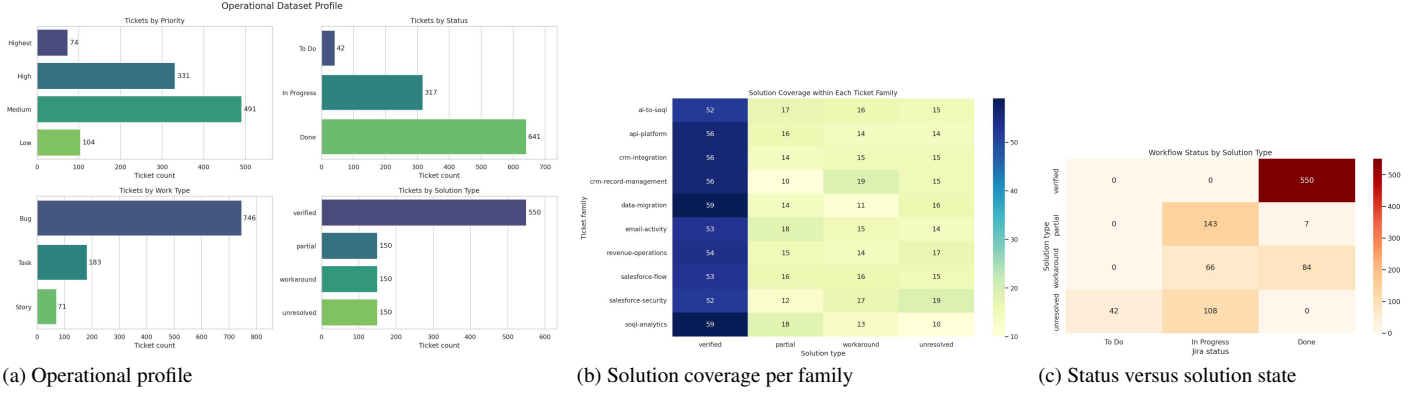


Fig. 1. Exploratory analysis of the approved corpus. The distribution is defect-heavy and mid-priority (a), every solution state is represented in every technical family (b), and workflow status is constrained by solution state (c): *verified* implies *Done*, *unresolved* is never *Done*, and the 84 *Done* *workaround* tickets are exactly the cases where a *Done* status is not proof of a fix.

entirely *Done* and the *unresolved* row is entirely open, while *partial* and *workaround* occupy the ambiguous middle that makes the task non-trivial — 84 *workaround* tickets are marked *Done* despite not being permanent fixes.

Critically, solution state and family are **evaluation labels**. They are never embedded, never placed in a prompt and never used as a QLoRA input; the exclusion is asserted at runtime, not merely documented.

D. Leakage-safe split

We split at *ticket* level, 75/15/10, seed 42, stratified jointly on family $\times$ solution type (Table I). All 40 strata are present in all three subsets, ticket IDs are pairwise disjoint, coverage is 1000/1000, and the test split is frozen by a manifest hash before any evaluation is allowed to run. Figure 2 shows the resulting proportions: train tracks the full corpus to within 0.1 percentage points, while validation and test drift by at most 3.0 points — the expected sampling noise of 150- and 100-ticket subsets, not a stratification defect.

E. Canonical RAG documents

Each ticket becomes exactly one deterministic document with four explicitly separated boundaries: **search text** (summary, component, description — the only text that is embedded), **answer content** (the fields shown to the generator), **operational meta-data** (status, priority, work type, component — filterable), and **evaluation metadata** (family, solution type, split — never embedded, never prompted). A fifth system block records the document version. Keeping the embedding surface and the generation surface distinct is what makes the leakage claim checkable rather than aspirational.

III. SYSTEM ARCHITECTURE AND DEPLOYMENT

A. Three paths, one vector store

Figure 3 shows the system as built. Three paths meet at a single persistent vector store, and keeping them separate is what makes the reproducibility and leakage claims checkable.

TABLE I. Stratified ticket-level split (seed 42), stratified on family $\times$ solution type.

Split	Tickets	%	Strata	Role
Train	750	75.0	40/40	QLoRA data, dev
Validation	150	15.0	40/40	All selection
Test	100	10.0	40/40	Frozen evidence
Total	1,000	100.0	40/40	0 overlapping IDs

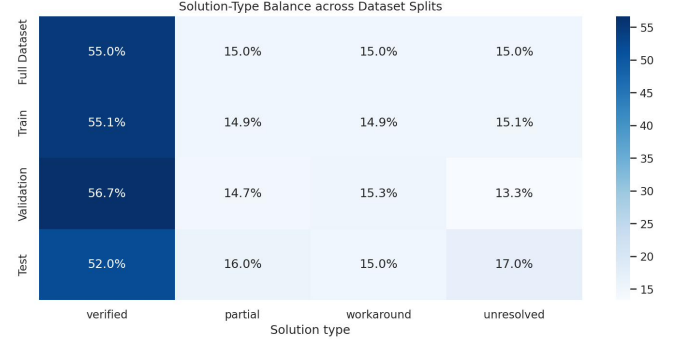


Fig. 2. Solution-type proportions across the full corpus and the three splits. Train reproduces the corpus to within 0.1 points; validation and test deviate by at most 3.0 points, consistent with their sample sizes.

The **offline build path** runs once and thereafter loads. The approved corpus passes the contract gate of Section II, becomes one canonical document per ticket with its four boundaries, and is encoded by multilingual E5 into the FAISS index. Evaluation labels are stripped at the document stage and never reach the encoder, which is asserted at runtime rather than assumed.

The **live overlay path** runs on demand. Issues labelled *jirag-demo* are fetched from Jira Cloud over REST v3, flattened from Atlassian Document Format to text, normalised into the same document schema, and hashed; only the delta is re-encoded and swapped into place atomically. The frozen 1,000-ticket index and every reported evaluation number are untouched by this path, which is why a live demonstration cannot contaminate the results.

The **answer path** runs per question. The agent router picks one of five deterministic routes, retrieval and reranking assemble at most five sources from the merged index, Gemma 4 with the selected adapter generates under Prompt v3, and the evidence gates decide whether the draft is returned, downgraded to a refusal, or withheld for human review. Generation is the last step and the easiest to suppress — the aggregation route never reaches it at all.

B. The Cladding Labs OS portal

The notebook is a research artifact; a manager should not be asked to open one. We therefore built a web portal, **Cladding Labs OS** [2], as the surface a non-technical user actually touches. It presents a hub of application tiles: a Jira tile that opens the live project board, and an Assistant tile that embeds the chat in an iframe. Tiles are role-aware — an employee account does not see

offline build • runs once, then LOADs

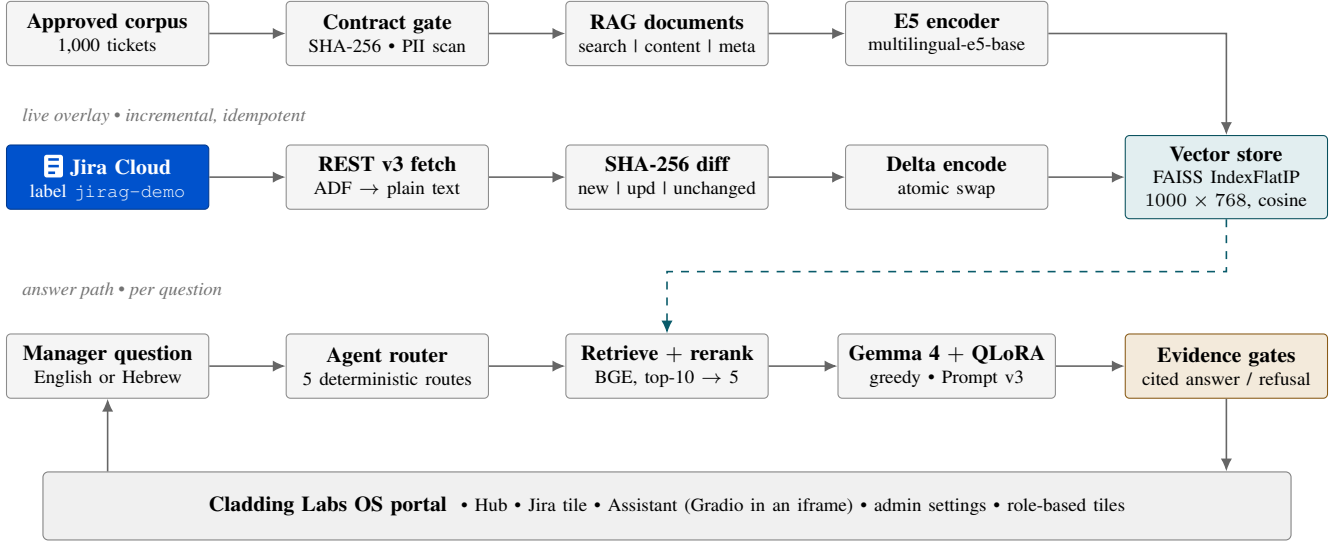


Fig. 3. jiRAG architecture. The offline build path (top) runs once and thereafter loads a fingerprinted artifact; the live overlay path (middle) re-encodes only changed Jira issues and merges them into the same store without disturbing the frozen corpus; the answer path (bottom) routes each question, retrieves and reranks evidence, generates under the grounding prompt and passes the draft through the evidence gates. The dashed edge is a read, not a write. Evaluation labels are excluded at the document stage and never reach the encoder or the prompt. Both the question and the cited answer travel through the Cladding Labs OS portal, which also links to the live Jira board.

the administrative tile that an administrator does — which is the minimum viable form of the role-based access control we list as future work.

The portal also absorbs an operational awkwardness. Because the assistant runs in a notebook behind a tunnel, its public URL changes on every restart, so the portal keeps that endpoint in an administrative setting alongside the Jira base and project URLs, and shows a connection indicator that turns green only when the backend actually answers. Separating the endpoint from the interface means the demonstration survives a runtime restart without touching the front end. It remains a demonstration: the model endpoint is unauthenticated and ephemeral, and stable hosting is future work rather than a claim.

IV. RETRIEVAL SYSTEM

A. Controlled embedding-model selection

The encoder is the single component that determines what the generator can ever see, so it was chosen by experiment rather than by reputation. We built a balanced train-only pilot of 80 documents (two per family $\times$ solution stratum) and 22 hand-written queries — 16 English and 6 Hebrew — and compared three encoders: `bge-base-en-v1.5` [3], `nomic-embed-text-v1.5` [4] and `multilingual-e5-base` [5]. Selection was *lexicographic*, not a weighted average: English Hit@5, then MRR, then Hit@3 and Hit@1, with the Hebrew slice consulted only on a tie.

The English metrics did tie (Table II, Fig. 4): BGE and E5 are identical at 93.8% Hit@1 and 0.969 MRR, and all three reach 100% English Hit@5. Hebrew broke the tie decisively — 83.3% Hit@1 for E5 against 16.7% and 0.0%. Cross-lingual retrieval, a Hebrew question locating an English ticket, therefore costs nothing in English quality and is the entire difference between an unusable and a usable system for Hebrew-speaking managers. We selected `intfloat/multilingual-e5-base`, 768-dimensional, with the `query:` and `passage:` prefixes it

TABLE II. Embedding pilot: 80 balanced train-only documents, 22 reviewed queries (16 EN, 6 HE). English ties; Hebrew decides.

Model	EN H@1	HE H@1	All H@1	MRR
<code>bge-base-en-v1.5</code>	93.8%	0.0%	68.2%	0.784
<code>nomic-embed-v1.5</code>	87.5%	16.7%	68.2%	0.742
<code>multilingual-e5-base</code>	93.8%	83.3%	90.9%	0.955

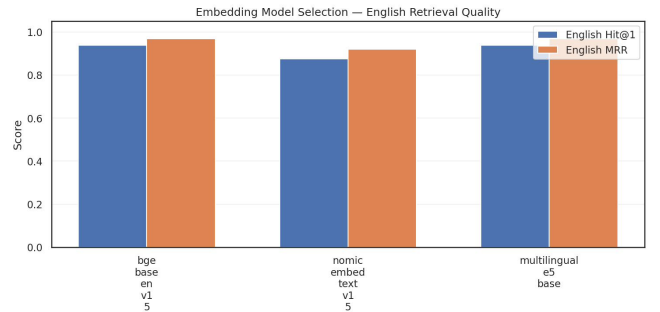


Fig. 4. English retrieval quality of the three candidate encoders. The English slice is effectively tied, which is precisely why the selection rule falls through to the Hebrew tie-breaker in Table II.

was trained under.

B. Persistent vector store

All 1,000 documents are encoded once into a 1000×768 `float32` matrix, L2-normalised, and indexed with a FAISS [6] `IndexFlatIP`, so inner product is cosine similarity by construction. At this corpus size an exhaustive flat index is exact and fast; approximate structures would trade correctness for a speed-up we do not need. The index, embedding matrix, ID mapping and a lineage manifest are persisted, and every reload asserts corpus count and split counts against the contract, `float32` dtype and unit norm to 10^{-5} , positional identity between FAISS vector i and document i , and file hashes against the manifest.

C. Five access paths, one schema

Semantic search is the interesting path, but a manager who types “ticket 186” or “how many High tickets are open” deserves an exact answer rather than a similar one. jiRAG therefore exposes five paths over one document schema: **semantic** (E5 query vector against the normalised index); **lookup**, where 186, ticket 186 and the Hebrew equivalent all normalise to `tckt-0186` with no embedding involved; **filter**, exact validated predicates over status, priority, work type and component; **aggregate**, deterministic Python counts with group-by, never generated — “open High-priority tickets” returns 92, broken down as 85 *In Progress* and 7 *To Do*; and **hybrid**, which filters first and ranks only the eligible subset, asserting afterwards that every returned hit satisfies the filter.

D. Cross-encoder reranking

The bi-encoder retrieves ten candidates, and a `bge-reranker-v2-m3` cross-encoder [7, 8] rescores them jointly with the query (batch size 16, maximum length 512), passing the best five forward as generation evidence. The distinction matters for what reranking can and cannot fix: it reorders a candidate set, so it can promote a Gold ticket sitting at rank 7 into the evidence window, but it can never recover a ticket that the bi-encoder failed to place in the top ten at all. Section VIII shows one instance of each.

V. GROUNDED GENERATION

A. Model and decoding

Generation uses `google/gemma-4-E4B-it` [9] in bf16 with **greedy decoding** — sampling disabled, so a fixed question and fixed evidence yield a fixed answer, which is a prerequisite for the evaluation protocol in Section VII rather than a stylistic preference. The input budget is 12,000 tokens with at most 320 generated tokens. When a source does not fit, its description is trimmed by binary search on token count rather than dropped, and the included, truncated and omitted ticket IDs are recorded alongside the answer, so an incomplete answer can always be traced to missing context rather than guessed at.

B. The grounding contract

The prompt is a versioned file whose SHA-256 enters the identity of every generated artifact, so an answer can never be silently produced by a different instruction set. Prompt v3 states five rules: answer strictly from the supplied tickets, treating retrieval rank as a hint and not as proof, and never merging merely similar incidents; answer every supported part of the question — what happened, whether it was resolved, cause, fix, validation, prevention — and state explicitly what cannot be determined; describe an outcome as permanent, partial, workaround or unresolved only where the text supports that wording, since *Done* alone is not a fix; cite each material claim locally as `tckt-XXXX`, using only IDs present in the context; and answer in the language of the question while never exposing scores, labels or reasoning.

Every answer is then checked programmatically: citations must be present, and every cited ID must belong to the retrieved set. Any citation outside that set marks the answer unsafe. Below is an unedited system answer from the baseline RAG path, abbreviated for space:

Q: *Were employees able to access sensitive supplier information, and if so, how was the exposure resolved?*

A: *Yes — a custom object containing commercial terms from a supplier agreement remained set to public read from a pilot [tckt-0186]. The exposure was resolved by setting the default access to private, adding a sharing rule for the procurement group and a criteria-based rule for approvers [tckt-0186]. Validation confirmed that users outside procurement received no results for the object in reports and search, while procurement retained full access [tckt-0186].*

VI. QLoRA FINE-TUNING

A. Configuration

Gemma is loaded under NF4 double quantization with bf16 compute; the base weights stay frozen and only low-rank adapters [10, 11] are trained. Both runs share: maximum sequence length 4,096 (observed median 1,766 tokens, maximum 1,949, zero truncated), 2 epochs, 398 optimizer steps, learning rate 2×10^{-4} with a cosine schedule and 3% warmup, per-device batch size 1 with gradient accumulation 4 (effective batch 4), paged AdamW 8-bit, LoRA dropout 0.05, gradient checkpointing and seed 42, on a single NVIDIA A100-SXM4-40GB. The **only** varied parameter is adapter rank, with alpha held at 2r: `qlora_r8` trains 2,269,184 parameters (0.029% of the model) and `qlora_r16` trains 4,538,368 (0.057%).

B. Leakage-safe distant supervision

Training data is built from train and validation documents only; test tickets are never seen. Each example pairs a templated question (one of three per solution state, chosen by hashing the ticket ID) with Prompt v3 and five context tickets — the source plus its four nearest same-split neighbours as hard distractors — and a target that opens with the solution state, continues with that ticket’s own RESOLUTION and VALIDATION sections, and closes with the citation. The supervision is therefore *distant*: no human wrote any target.

Train contains 794 examples (750 single-source, 22 multi-source, 22 no-answer) and validation 158 (150/4/4). Integrity gates assert that train and validation are disjoint, that Gold sources appear in context, that target citations are exact, and that evaluation labels are excluded from inputs. Loss is computed on assistant tokens only; the prompt boundary is located by comparing the tokenized prompt against the tokenized full chat, and any mismatch inside the prompt aborts training rather than silently mislabelling the mask.

C. Convergence

Both adapters converged cleanly over the two-epoch schedule (Fig. 5): training loss falls from roughly 1.3 to near zero within about 100 steps and stays flat, and validation loss decreases monotonically across both epochs. `qlora_r16` reaches the lower language-model loss on both curves — a fact that, as Section VIII shows, did *not* decide the selection.

VII. EVALUATION PROTOCOL

A. Benchmark

The benchmark is 44 hand-written questions with Gold ticket IDs and expected answer points: 24 on validation (20 single-ticket, 2 multi-ticket, 2 no-answer) and 20 on the frozen test split (16/2/2). Eight questions across both splits are in Hebrew and target English tickets. No-answer questions describe plausible incidents that do not exist in the corpus, so the correct response is a refusal. The loader proves that every Gold ID belongs to the split the question is evaluated on.

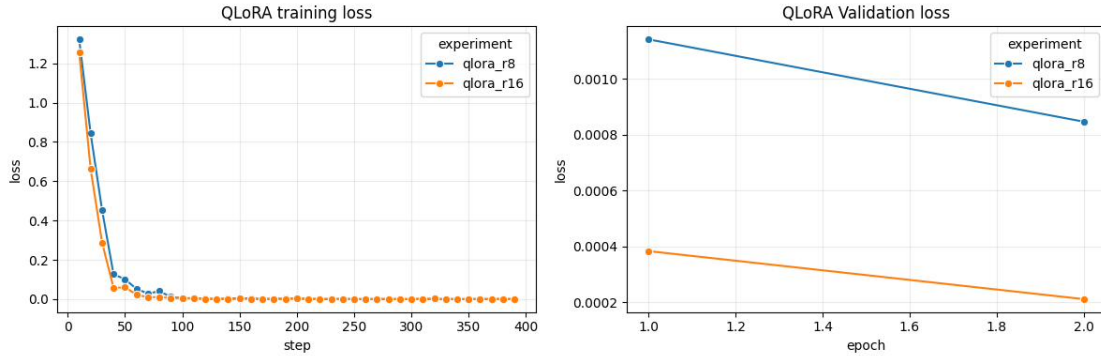


Fig. 5. QLoRA training loss per optimizer step (left) and validation loss per epoch (right) for the two controlled runs. Rank and proportional alpha are the only difference between them. Both converge; `qlora_r16` attains the lower loss on both curves, but language-model loss measures token prediction, not the grounding behaviour the system is selected on.

B. Metrics, kept separate on purpose

Four families of metric are reported separately because they measure different properties and are misleading when averaged. **Retrieval:** Hit@1, Hit@5, Recall@5, Complete@5 and MRR@5 against Gold IDs. **Citations:** presence, all-from-context, Gold recall and Gold precision. **Content:** expected-point coverage, where a point counts as covered when its E5 similarity to the answer is at least 0.75, plus ROUGE-L [12]. **Safety:** no-answer abstention, invented-citation rate and language match. Expected-point coverage and ROUGE-L measure semantic overlap and do not establish that a claim is grounded; they are never read without citation recall beside them.

C. The freeze

Selection of the retriever, the prompt and the adapter happens on validation only. The chosen configuration is then written to disk as a fingerprint over retriever, candidate k , output k , prompt version, generation model, benchmark version and similarity threshold; the file is re-read and verified before any test cell will execute, and a mismatch raises a `RuntimeError` rather than producing a number. Test results were used for reporting, not for choosing a rank, a prompt or a retriever, and nothing was re-tuned after they were seen. Section X contains the cost of honouring that rule.

VIII. RESULTS AND DISCUSSION

A. Retrieval and reranking

Reranking helps where the theory says it should (Table III). On validation it lifts Hit@1 by 9 points (0.818 $\rightarrow$ 0.909) and MRR@5 by 0.068, and it recovers `tckt-0673` for a multi-ticket question the bi-encoder had missed entirely. Its sibling source `tckt-0467` never entered the top-10 candidate set, so no reranker could recover it, and Complete@5 is consequently unchanged at 0.909 — the clearest possible illustration that reranking fixes *order*, not *recall*. On the frozen test split, every Gold ticket the baseline had found is moved to rank 1, giving Hit@1 1.000 and MRR@5 1.000; the single residual miss, a second source in `test-multi-001`, is again absent from the candidate set.

B. Prompt engineering

Replacing the baseline prompt with v3 on the same 24 validation questions raises citation presence from 0.955 to 1.000, all-points-covered from 0.909 to 0.955, expected-point coverage from 0.943 to 0.966 and Gold citation recall from 0.864 to 0.886, while all-from-context citations and no-answer abstention hold at 1.000. Gold citation precision drops slightly, 0.905 $\rightarrow$ 0.886: the in-

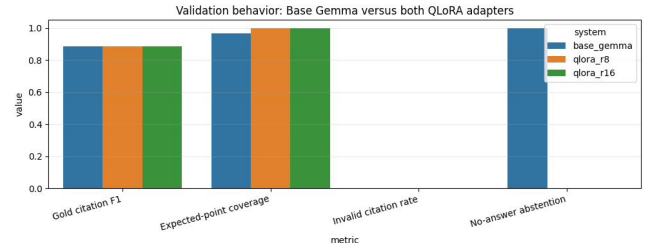


Fig. 6. Validation behaviour of base Gemma and the two adapters. Citation F1 and invalid-citation rate are identical across all three systems; coverage improves marginally under fine-tuning; and the only visible separation is the fourth group, where both adapters lose the abstention the base model retains.

struction to answer every supported part of the question makes the model cite more sources, some of which are relevant but not Gold. That is the expected signature of higher recall, and for an assistant whose failure mode is under-evidencing, we judged the trade acceptable.

C. Adapter selection

The two adapters were compared on the same 24 validation questions, the same reranked evidence and the same prompt (Table IV, Fig. 6). The selection order is deliberately ordered by grounding risk: minimise invalid citations, then preserve abstention, then maximise citation F1, then maximise coverage. Both adapters reach zero invalid citations, identical Gold citation F1 of 0.886, and identical perfect expected-point coverage — a four-way tie — and both lose no-answer abstention entirely. `qlora_r8` therefore wins the tie by being half the size. Note what this does *not* depend on: `qlora_r16` had the lower language-model loss in Fig. 5 and lost anyway, because loss measures token prediction and the system is selected on behaviour.

TABLE III. Retrieval before and after cross-encoder reranking, over answerable questions (22 of 24 validation, 18 of 20 test).

System	H@1	H@5	R@5	C@5	MRR@5
<i>Validation (selection split)</i>					
E5 $\rightarrow$ FAISS	0.818	0.909	0.909	0.909	0.864
+ BGE rerank	0.909	0.955	0.932	0.909	0.932
<i>Test (frozen)</i>					
E5 $\rightarrow$ FAISS	0.944	1.000	0.972	0.944	0.972
+ BGE rerank	1.000	1.000	0.972	0.944	1.000

TABLE IV. Validation behaviour: base Gemma versus both adapters, identical questions, evidence and prompt.

Metric	Base	r=16	r=8
Invalid citation rate	0.000	0.000	0.000
No-answer abstention	1.000	0.000	0.000
Gold citation F1	0.886	0.886	0.886
Expected-point coverage	0.966	1.000	1.000
All points covered	0.955	1.000	1.000
Multi-source cit. recall	0.750	0.750	0.750
ROUGE-L	0.285	0.325	0.322

TABLE V. Controlled 2×2 ablation on the frozen test split. Same 20 questions and prompt; only retrieval and the adapter vary.

Metric	No RAG		With RAG	
	Base	r=8	Base	r=8
Gold citation recall	0.000	0.000	0.972	0.944
Invented citation rate	0.250	0.650	0.000	0.000
Expected-point coverage	0.972	0.722	0.986	1.000
All points covered	0.889	0.444	0.944	1.000
No-answer abstention	1.000	1.000	1.000	0.000
Answerable abstention	0.500	0.222	0.000	0.000
ROUGE-L	0.118	0.058	0.262	0.321

D. The 2×2 ablation

The four systems in Table V and Fig. 7 share the same 20 frozen test questions and the same Prompt v3; only retrieval and the adapter vary. The instructive quantities are the arrows, not the cells.

Cell 1 → cell 3 isolates retrieval. Gold citation recall moves from 0.000 to 0.972 and the invented-citation rate from 0.25 to 0.00. Without retrieval the base model answers 10 of 20 questions from nothing and abstains on the other half, and a quarter of what it does produce cites ticket IDs it was never shown. Retrieval, not fine-tuning, is the grounding mechanism.

Cell 3 → cell 4 isolates fine-tuning. Expected-point coverage rises 0.986 → 1.000, all-points-covered 0.944 → 1.000, citation precision 0.972 → 1.000 and ROUGE-L 0.262 → 0.321, with invented citations still at zero. The adapter makes the system answer every part of a multi-part question, which the base model tends to leave half-finished. But no-answer abstention collapses from 1.000 to 0.000 and multi-source citation recall halves, 0.750 → 0.500.

Cell 2 is the diagnostic. With the adapter and no retrieval, 65% of answers invent a citation in exactly the [tckt-XXXX] format the adapter was trained on, expected-point coverage falls to 0.722 and ROUGE-L to 0.058. The adapter learned the *shape* of a grounded answer — opening sentence, resolution, validation, bracketed ID — and none of the ticket facts. That is the desired outcome for a leakage argument: nothing transferred from the corpus into the weights in a usable form. It is also why expected-point coverage is never reported alone; cell 1 scores 0.972 on coverage while citing nothing real.

E. System decision and its honest reading

QLoRA + RAG is the selected primary answer path: it is the most complete system (all expected points covered on every answerable question) and the most precisely cited (citation precision 1.000, zero invented citations). The choice is explicitly conditional. Base + RAG remains the safer generator on unsupported

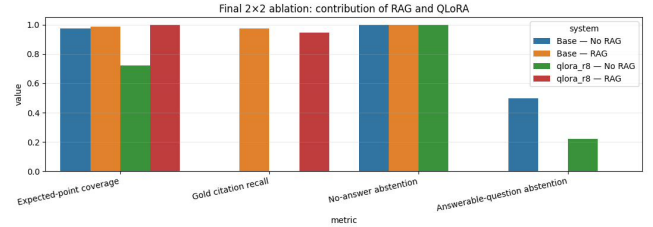


Fig. 7. Frozen-test 2×2 ablation. Gold citation recall is non-zero only in the two retrieval conditions — the single clearest statement in the project. Answerable-question abstention, rightmost, shows the complementary failure: without evidence the base model refuses half the answerable questions rather than inventing an answer.

questions and retains three quarters of Gold sources on the multi-ticket slice against the adapter’s half, so the selected generator is deployed only behind the agent evidence gate of Section IX. Neither no-retrieval cell is a deployment option at all.

The near-ceiling test retrieval numbers deserve scepticism rather than celebration. A synthetic corpus gives every ticket a distinct summary, and 16 of the 18 answerable test questions are single-ticket; Hit@5 = 1.000 on 18 questions is a statement about this corpus, not about Jira. The slices that actually stress the system are the multi-ticket questions, where recall sits between 0.500 and 0.750, and the Hebrew slice, which held at rank 1 through both retrieval stages but generates less naturally than English.

IX. AGENT GUARDRAILS AND LIVE OPERATION

A. Deterministic routing

Every turn produces an explicit plan before any tool runs — route, ticket ID, filters, whether generation is needed, advisory level — and the plan is returned with the response, so a wrong answer can be traced to a wrong decision rather than to an opaque model. Routing is keyword-based over five routes (Table VI). This is transparent and testable, and it breaks on misspellings and indirect phrasing; it is not an LLM planner and we do not describe it as one.

B. Evidence gates

Four gates apply in order. A top semantic score below 0.70 marks the evidence insufficient and suppresses generation. A generator that abstains on its own downgrades the state to insufficient. An answer claiming a resolution about a cited *unresolved* or *workaround* ticket is withheld for human review — the contract rule from Section II doing its work. And any citation outside the retrieved set sets `citation_safe` to false.

The first gate is what restores the abstention the adapter lost. On the harness scenario `new-no-answer-001` the selected system is stopped before generation and returns a refusal. We record this as a *guardrail*, not a fix: the underlying cause is 22 refusal examples in 794, and repairing it properly means adding

TABLE VI. Agent routes. Aggregation never generates: counts are deterministic Python.

Route	Trigger	Generates?
lookup	explicit ticket ID	only for why/how
aggregation	“how many” / count	never
hybrid	topic + priority or open	yes
semantic	everything else	yes
similarity	new-ticket report	as history only

refusal data and re-running the same selection rule, not patching after the fact.

A seven-scenario development harness (one in Hebrew, none drawn from the benchmark) passes all route, evidence, citation, advisory and no-write assertions at 1.000. That is a test of the rules, not a language-quality score; manual inspection still finds occasional status-phrasing inconsistencies.

C. Read-only by construction

The advisory writer can recommend that a manager confirm a preventive control is still active. It cannot transition, edit or create an issue: `write_executed` is hard-coded false, and “Move `tckt-0186` to Done” returns a refusal. Historical incidents are surfaced as investigation leads, never as proof of a new report’s root cause.

D. Incremental Jira synchronization

The live extension reads Jira Cloud REST v3 for issues labelled `jirag-demo`, capped at 50 for the proof of concept. Descriptions arrive as Atlassian Document Format and are flattened to text, normalised into the same document schema as the corpus, and hashed; a SHA-256 diff against the last manifest classifies each issue as new, updated, unchanged or removed, and only the delta is re-encoded. Documents, embeddings, mapping, index and manifest are staged in a temporary directory and swapped atomically. Live documents receive IDs from `tckt-1001` upward so they can be cited like any other source, and exact lookup accepts research IDs, live IDs and Jira keys alike. A second synchronization with nothing edited reports two issues fetched, zero new, zero updated and zero encoded — the idempotency check. The frozen 1,000-ticket index and all reported evaluation results are untouched by this overlay.

A Gradio [13] chat merges the frozen index with the live overlay and exposes a source table, a synchronized-issue table and explicit *Sync* and *Clear* actions, so context boundaries are the user’s decision rather than a hidden heuristic. That chat is what the portal of Section B embeds, so a manager creating a Jira issue and a manager asking about it are working in the same browser tab.

X. LIMITATIONS

Abstention. The fine-tuned model answered both no-answer test questions. With two such questions per split, the metric is 0, 0.5 or 1 with nothing in between, so the design is also under-powered to measure what it reports.

Synthetic corpus. Every ticket has a distinct summary and a complete seven-section body. Real Jira has duplicates, one-line descriptions and screenshots instead of text. The retrieval numbers characterise this corpus.

Template supervision. QLoRA targets were derived from each ticket’s own sections rather than written by a human, which straightforwardly explains the ROUGE-L gain. The behavioural gain does transfer to human-written test questions, but the adapter has never seen a human answer.

Keyword routing and Hebrew. The router breaks on misspellings and indirect phrasing; Hebrew generation is less natural than English, and the abstention phrase list had to be maintained in both languages by hand.

Testing. Validation is by runtime assertion, of which there is a great deal. The pure functions — ID normalisation, citation validation, ADF flattening, routing, loss masking — deserve

unit tests and do not yet have them. Deployment is a demonstration rather than hosting: the Cladding Labs OS portal is a real role-aware front end, but the assistant endpoint behind it is an unauthenticated Gradio share URL that dies with the runtime.

XI. CONCLUSION AND FUTURE WORK

We built a complete, reproducible retrieval-augmented system over 1,000 contract-validated Jira tickets that answers from retrieved evidence, cites every material claim, and is read-only by construction. Selecting a multilingual encoder by controlled experiment lifted overall Hit@1 from 0.682 to 0.909 and made Hebrew retrieval possible at all; cross-encoder reranking then lifted validation Hit@1 by 9 points and drove frozen-test MRR@5 to 1.000 while leaving Complete@5 unchanged, precisely because reranking cannot recover what retrieval never proposed.

The 2×2 ablation answers the question the project was designed around. **Retrieval supplies the grounding:** it is the only difference between Gold citation recall 0.000 and 0.972, and between a 25% invented-citation rate and none. **QLoRA changes how that evidence is used,** raising coverage to 1.000, citation precision to 1.000 and ROUGE-L by 0.059 — and simultaneously erasing the refusal behaviour that made the base model safe on unanswerable questions. Fine-tuning for style silently eroded a safety property, 22 refusal examples did not hold the line, and the honest response was to report it and restore refusal in a deterministic gate rather than to re-tune against a test set we had already opened.

Future work follows directly from the limitations. Adding 100 or more refusal examples and re-running the identical selection rule would test whether the abstention collapse is a data-quantity problem, as we believe. Replacing the keyword router with a small classifier addresses the typo brittleness; an entailment or evidence-judge model would strengthen multi-source verification, where the selected adapter cites one of two Gold tickets half the time. Beyond that: unit tests for the pure functions, stable hosting with an authenticated API behind role-based access control, and — the experiment that would say the most — running the same frozen pipeline against a real, messy Jira export.

Reproducibility. All code, the Colab notebook, the dataset contract, the versioned prompts, the full metric reports and the presentation deck are in the project repository [14, 15]; the manager-facing portal is at <https://cladding-labs-os.com> [2].

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] I. Abramovich, S. Bondarenko, and O. Ohayon, “Cladding Labs OS: Manager-facing portal for the jiRAG assistant.” <https://cladding-labs-os.com>, 2026.
- [3] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packed resources for general Chinese embeddings,” *arXiv preprint arXiv:2309.07597*, 2023.
- [4] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” *arXiv preprint arXiv:2402.01613*, 2024.
- [5] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual E5 text embeddings: A technical report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [6] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [7] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embed-

- dings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [8] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” *arXiv preprint arXiv:1901.04085*, 2019.
 - [9] Gemma Team, Google DeepMind, “Gemma: Open models based on Gemini research and technology.” <https://ai.google.dev/gemma>, 2024. Checkpoint used in this work: `google/gemma-4-E4B-it`.
 - [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
 - [11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
 - [12] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out, ACL Workshop*, pp. 74–81, 2004.
 - [13] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-free sharing and testing of ML models in the wild,” *arXiv preprint arXiv:1906.02569*, 2019.
 - [14] I. Abramovich, S. Bondarenko, and O. Ohayon, “jiRAG: Jira retrieval-augmented generation assistant — source repository.” https://github.com/Sagitb/rag_project_google_reichman, 2026.
 - [15] I. Abramovich, S. Bondarenko, and O. Ohayon, “jiRAG: Executable Colab notebook and presentation deck.” https://colab.research.google.com/drive/1r_cEVbmhRfOfjKQ_d021mrV7Zcip_c0u, 2026.